TP: Breaking the Classics

A Cryptography Escape Room

Intro to Cryptography

Name: Mohamed AIT LAHCEN

Date: 11 October 2026

Warm-Up

Exercise 1 — Caesar encryption

Question: Encrypt "CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ" with K = 4.

Code

```
def caesar(text, k):
    return "".join(chr((ord(c) - 65 + k) % 26 + 65) if c.isalpha() else c for c in text.upper())

plaintext = "CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ"
K = 4
print("Plaintext :", plaintext)
print("Key       :", K)
print("Ciphertext:", caesar(plaintext, K))
```

Output

```
Plaintext : CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ
Key       : 4
Ciphertext: GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD
```

Answer: GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD

Exercise 2 — Caesar decryption

Question: Decrypt the message encrypted with K = 8.

Code

```
def caesar(text, k):
    return "".join(chr((ord(c) - 65 + k) % 26 + 65) if c.isalpha() else c for c in text.upper())

ciphertext = "GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA"
K = 8
print("Ciphertext:", ciphertext)
print("Key       :", K)
print("Plaintext :", caesar(ciphertext, -K))
```

Output

```
Ciphertext: GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA
Key       : 8
Plaintext : YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS
```

Answer: YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS

Exercise 3 — Brute force

Question: Recover the plaintext without the key.

Code

```
def caesar(text, k):
    return "".join(chr((ord(c) - 65 + k) % 26 + 65) if c.isalpha() else c for c in text.upper())

def brute_force(ciphertext):
    print("Ciphertext:", ciphertext, "\n")
    for key in range(26):
        print(f"Key {key:2d} : {caesar(ciphertext, -key)}")

brute_force("ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP")
```

Output

```
Ciphertext: ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP

Key  0 : ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP
Key  1 : DRO UOI SC EXUXYGX DBI LBEDO PYBMO
Key  2 : CQN TNH RB DWTWXFW CAH KADCN OXALN
Key  3 : BPM SMG QA CVSVWEV BZG JZCBM NWZKM
Key  4 : AOL RLF PZ BURUVDU AYF IYBAL MVYJL
Key  5 : ZNK QKE OY ATQTUCT ZXE HXAZK LUXIK
Key  6 : YMJ PJD NX ZSPSTBS YWD GWZYJ KTWHJ
Key  7 : XLI OIC MW YRORSAR XVC FVYXI JSVGI
Key  8 : WKH NHB LV XQNQRZQ WUB EUXWH IRUFH
Key  9 : VJG MGA KU WPMPQYP VTA DTWVG HQTEG
Key 10 : UIF LFZ JT VOLOPXO USZ CSVUF GPSDF
Key 11 : THE KEY IS UNKNOWN TRY BRUTE FORCE
Key 12 : SGD JDX HR TMJMNVM SQX AQTSD ENQBD
Key 13 : RFC ICW GQ SLILMUL RPW ZPSRC DMPAC
Key 14 : QEB HBV FP RKHKLTK QOV YORQB CLOZB
Key 15 : PDA GAU EO QJGJKSJ PNU XNQPA BKNYA
Key 16 : OCZ FZT DN PIFIJRI OMT WMPOZ AJMXZ
Key 17 : NBY EYS CM OHEHIQH NLS VLONY ZILWY
```

```
Key 18 : MAX DXR BL NGDGHPG MKR UKNMX YHKVX
Key 19 : LZW CWQ AK MFCFGOF LJQ TJMLW XGJUW
Key 20 : KYV BVP ZJ LEBEFNE KIP SILKV WFITV
Key 21 : JXU AUO YI KDADEMD JHO RHKJU VEHSU
Key 22 : IWT ZTN XH JCZCDLC IGN QGJIT UDGRT
Key 23 : HVS YSM WG IBYBCKB HFM PFIHS TCFQS
Key 24 : GUR XRL VF HAXABJA GEL OEHGR SBEPR
Key 25 : FTQ WQK UE GZWZAIZ FDK NDGFQ RADOQ
```

Answer: Key 11: THE KEY IS UNKNOWN TRY BRUTE FORCE. Caesar has only 26 keys, so brute force is trivial.

Room 1: The Letter Wall

Exercise 1 — Substitution encryption

Question: Encrypt "KEEP THE SECRET SAFE" with the given alphabet.

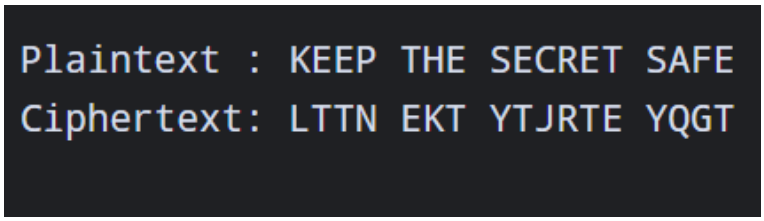
Code

```
plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QMJZTGFKPWLSBOXNCRYEVHIADU"

def encrypt(text):
    table = dict(zip(plain, cipher))
    return "".join(table.get(c, c) for c in text.upper())

message = "KEEP THE SECRET SAFE"
print("Plaintext :", message)
print("Ciphertext:", encrypt(message))
```

Output



Answer: LTTN EKT YTJRTE YQGT

Exercise 2 — Substitution decryption

Question: Decrypt "ERVYE DXVR SXFPJ" with the same alphabet.

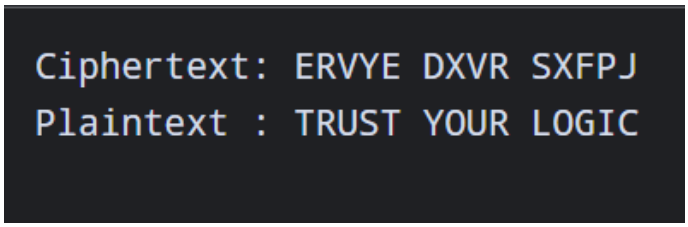
Code

```
plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QMJZTGFKPWLSBOXNCRYEVHIADU"

def decrypt(text):
    table = dict(zip(cipher, plain))
    return "".join(table.get(c, c) for c in text.upper())

ciphertext = "ERVYE DXVR SXFPJ"
print("Ciphertext:", ciphertext)
print("Plaintext :", decrypt(ciphertext))
```

Output



Answer: TRUST YOUR LOGIC (clue for the next room).

Exercise 3 — Frequency analysis

Question: How can an attacker use letter frequencies? Write a frequency-analysis program.

Answer: The most frequent ciphertext letters are matched to the most frequent English letters (E, T, A, O...), which gives part of the key. Short words and repeated patterns then complete it.

Code

```
from collections import Counter

def frequency_analysis(text):
    letters = [c for c in text.upper() if c.isalpha()]
    return len(letters), Counter(letters).most_common()

def show(text):
    total, freq = frequency_analysis(text)
    print(f"Total letters: {total}\n")
    print("Letter  Count  Percent")
    for letter, n in freq:
        print(f"  {letter}     {n:3d}   {100 * n / total:5.2f}%")

ciphertext = """EKT YTJRTE BTYYQFT PY KPZZTO PO EKPY NQRQFRQNK.
STEETR GRTCVTOJD ZXTY OXE RTHTQS EKT IKXST LTD,
MVE PE FPHTY Q FXXZ YEQRE.
SXXL GXR JXBBXO IXRZY, RTNTQETZ NQEETROY,
QOZ YKXRE IXRZY.
IKTO DXV RTJXHTR EKT BTYYQFT,
EKT HPFTOTRT LTD KQY GXVR STEETRY."""

show(ciphertext)
```

Output

```
Total letters: 197

Letter  Count  Percent
  T      34   17.26%
  E      19    9.64%
  R      18    9.14%
  Y      16    8.12%
  X      16    8.12%
  Q      12    6.09%
  K      11    5.58%
  O       9    4.57%
  Z       8    4.06%
  P       7    3.55%
  F       6    3.05%
  S       5    2.54%
  J       4    2.03%
  B       4    2.03%
  N       4    2.03%
  V       4    2.03%
  D       4    2.03%
```

```
  H       4    2.03%
  I       4    2.03%
  G       3    1.52%
  L       3    1.52%
  C       1    0.51%
  M       1    0.51%
```

Answer: T (17.26%) ≈ E, E (9.64%) ≈ T, R (9.14%) ≈ R.

Exercise 4 — Break the intercepted message

Question: Recover the plaintext and the clue for the next room.

Code

```
from collections import Counter

ciphertext = """EKT YTJRTE BTYYQFT PY KPZZTO PO EKPY NQRQFRQNK.
STEETR GRTCVTOJD ZXTY OXE RTHTQS EKT IKXST LTD,
MVE PE FPHTY Q FXXZ YEQRE.
SXXL GXR JXBBXO IXRZY, RTNTQETZ NQEETROY,
QOZ YKXRE IXRZY.
IKTO DXV RTJXHTR EKT BTYYQFT,
EKT HPFTOTRT LTD KQY GXVR STEETRY."""

def apply(mapping, text):
    return "".join(mapping.get(c, "_") if c.isalpha() else c for c in text)

letters = [c for c in ciphertext if c.isalpha()]
ranking = [l for l, _ in Counter(letters).most_common()]
print("Cipher letters, most -> least frequent:\n", " ".join(ranking), "\n")

stage1 = {"T": "E", "E": "T", "R": "R", "Y": "S", "X": "O", "Q": "A",
          "K": "H", "O": "N", "Z": "D", "P": "I", "F": "G", "S": "L"}
print("--- Stage 1 (frequency guess) ---")
print(apply(stage1, ciphertext), "\n")

final = dict(stage1)
final.update({"J": "C", "B": "M", "N": "P", "I": "W", "L": "K",
              "G": "F", "H": "V", "D": "Y", "V": "U", "M": "B", "C": "Q"})
print("--- Final plaintext ---")
print(apply(final, ciphertext), "\n")

print("Mapping (cipher -> plain):")
for c in sorted(final):
    print(f"  {c} -> {final[c]}")
```

Output

```
Cipher letters, most -> least frequent:
 T E R Y X Q K O Z P F S J B N V D H I G L C M

--- Stage 1 (frequency guess) ---
THE SE_RET _ESSAGE IS HIDDEN IN THIS _ARAGRA_H.
LETTER _RE__EN__ DOES NOT RE_EAL THE _HOLE _E_,
__T IT GI_ES A GOOD START.
LOO_ _OR _O__ON _ORDS, RE_EATED _ATTERNS,
AND SHORT _ORDS.
_HEN _O_ RE_O_ER THE _ESSAGE,
THE _IGENERE _E_ HAS _O_R LETTERS.

--- Final plaintext ---
THE SECRET MESSAGE IS HIDDEN IN THIS PARAGRAPH.
LETTER FREQUENCY DOES NOT REVEAL THE WHOLE KEY,
BUT IT GIVES A GOOD START.
LOOK FOR COMMON WORDS, REPEATED PATTERNS,
AND SHORT WORDS.
WHEN YOU RECOVER THE MESSAGE,
THE VIGENERE KEY HAS FOUR LETTERS.
```

```
Mapping (cipher -> plain):
  B -> M
  C -> Q
  D -> Y
  E -> T
  F -> G
  G -> F
  H -> V
  I -> W
  J -> C
  K -> H
  L -> K
  M -> B
  N -> P
  O -> N
  P -> I
  Q -> A
  R -> R
  S -> L
  T -> E
```

Answer: THE SECRET MESSAGE IS HIDDEN IN THIS PARAGRAPH. Clue: the Vigenère key has four letters.

Room 2: Break the Vigenère Cipher

Exercise — Recover the key, decrypt, find the access word

Question: Break the Vigenère ciphertext (key length 4).

Code

```
from collections import Counter

ciphertext = ("VVHWGQUIVYHIRSUPGTWXJSVIPCWIUPHGCIVIVVHPGHWITSDTRSDVUSYITMZL"
              "GFHIXSUCVSDQOSPFGFVLQIOHEVHGMSDGJQRPWAQWGSNXJSPSUHIVGEXIPHVC"
              "OPRPCBGYUSLXVCUIECYITHKIMSBAJSQXJSPIUGDKGPHGQAHWEZHETSQXGFWL"
              "GOFGGGVAQFGIEZLTUS")

def recover_key(text, length):
    key = ""
    for i in range(length):
        group = text[i::length]
        counts = Counter(group)
        top = counts.most_common(1)[0][0]
        shift = (ord(top) - ord("E")) % 26
        key += chr(shift + 65)
        print(f"Group {i + 1}: {group}")
        print("   top 5 letters:", counts.most_common(5))
        print(f"   most frequent = {top}  ->  shift {shift}  ->  key letter {key[-1]}\n")
    return key

def decrypt(text, key):
    return "".join(chr((ord(c) - ord(key[i % len(key)])) % 26 + 65) for i, c in enumerate(text))

key = recover_key(ciphertext, 4)
print("Recovered key:", key)
print("\nPlaintext:\n")
print(decrypt(ciphertext, key))
```

Output

```
Group 1: VGVRGJPUCVGTRUTGXVOGQEMJWGJUGPOCUVETMJJUGQETGGGQEU
   top 5 letters: [('G', 11), ('U', 6), ('V', 5), ('J', 5), ('T', 4)]
   most frequent = G  ->  shift 2  ->  key letter C

Group 2: VQYSTSCPIVHSSSMFSSSFIVSQASSHEHPBSCCHSSSGPAZSFOGFZS
   top 5 letters: [('S', 17), ('H', 4), ('F', 4), ('V', 3), ('C', 3)]
   most frequent = S  ->  shift 14  ->  key letter O

Group 3: HUHUWVWHVHWDDYZHUDPVOHDRQNPIXVRGLUYKBQPDHHHQWFVGL
   top 5 letters: [('H', 9), ('V', 5), ('D', 5), ('U', 4), ('W', 4)]
   most frequent = H  ->  shift 3  ->  key letter D

Group 4: WIIPXIIGIPITVILICQFLHGGPWXSVICPYXIIIAXIKGWEXLGAIT
   top 5 letters: [('I', 14), ('X', 5), ('G', 5), ('P', 4), ('W', 3)]
   most frequent = I  ->  shift 4  ->  key letter E

Recovered key: CODE
```

```
Plaintext:

THESECRETKEEPERLEFTTHESENOTESBECAUSETHELETTEREAPPEARSEVERYWHEREEVERYTEAMMEMBERSHOULDCHECKE
ACHCOLUMNSEEKTHEMOSTFREQUENTSYMBOLANDUSEITTORECOVERTHEKEYWHENTHEMESSAGEBECOMESCLEARENTERTH
EACCESSWORDECLIPSE
```

Answer: Key = **CODE**. Access word = **ECLIPSE**.

Caesar falls to brute force (26 keys). Substitution has 26! keys but frequency analysis breaks it. Vigenère reduces to four Caesar ciphers once the key length is known.